MAIN SCRIPT-

“I built **CodeSync**, a real-time collaborative coding platform where multiple users can join a room and write code together, similar to Google Docs but for coding.

The main goal was to enable **low-latency collaboration**, so I used **WebSockets via Socket.io** to sync code changes instantly across all users in a room.

The frontend is built using React, and the backend uses Node.js with Express. I designed a **room-based architecture**, where each room maintains its own code state and connected users.

Whenever a user types, the changes are emitted to the server and broadcast to other users in the same room, ensuring real-time synchronization.

I also implemented backend APIs for session management and code persistence using MongoDB.

Additionally, I extended the platform with features like an AI assistant for debugging and multi-language code execution.”

Q1: “How is CodeSync different from existing platforms?”

“Platforms like Google Docs or Visual Studio Code Live Share already support collaboration, but CodeSync is designed as a **focused, all-in-one environment for coding collaboration**.

Unlike generic editors, CodeSync combines:

- real-time code collaboration
- in-room chat and AI debugging assistance
- multi-language code execution

all inside a single platform without needing external tools.

Compared to tools like VS Code Live Share, which require setup and extensions, my platform is **browser-based and instantly accessible**, making it easier for interviews and quick collaboration.

Also, my focus was on **learning and interview use cases**, where AI assistance and execution features are tightly integrated into the workflow.”

Q2. “Explain how real-time code synchronization works in your system.”

“Real-time synchronization in my system is handled using WebSockets via Socket.io, not traditional REST APIs.

When a user types in the editor, a **code change event** is emitted from the frontend through a WebSocket connection to the server.

The server then receives this event and **broadcasts it to all other users in the same room** using Socket.io’s room-based communication.

On the receiving side, each client listens for these events and updates the editor state accordingly, ensuring that all users see changes in real time.”

Q3: How did you design your room system? How are users managed in a room?

“In my system, each room represents a collaborative session where multiple users can join and interact in real time.

On the backend, I maintain a room structure where each room has a unique roomId and a list of active users.

When a user joins, they emit a `join_room` event with their username and roomId. The server then:

- adds the user to that room using Socket.io’s room feature
- updates the room’s user list
- notifies other users in the room about the new participant

Similarly, when a user disconnects or leaves, the server removes them from the room and broadcasts the updated user list.

This ensures that all users in a room always have a synchronized view of who is currently active.”

“I use Socket.io rooms for real-time communication and MongoDB for persistence.

When a user joins a room:

- They are added to a Socket.io room for real-time event broadcasting
- At the same time, I update MongoDB to store room data like users and code for persistence

So Socket.io handles the live collaboration, while MongoDB ensures data is not lost and can be restored later."

Q4: What happens when two users type at the same time? How do you handle conflicts?

"When two users type at the same time, both clients emit code change events to the server using Socket.io.

The server processes these events in the order they are received and broadcasts them to all users in the room.

Currently, I handle conflicts using a **last-write-wins strategy**, meaning the most recent update overwrites the previous one.

This works well for small-scale collaboration, but I'm aware that more advanced systems use techniques like **CRDTs or Operational Transformation** to merge changes more intelligently."

❓ "How does server decide which is latest?"

Say:

"The server processes events sequentially as they arrive over the WebSocket connection, so the order of arrival determines which update is applied last."

❓ "What's the drawback of last-write-wins?"

Say:

"It can overwrite intermediate changes, so some user edits might get lost if multiple users type at the same time."

❓ "How would you improve it?"

Say:

"I would implement CRDT-based syncing, which allows concurrent edits without overwriting changes."

Q5: How does your AI assistant work? What happens internally when a user asks a question?

"My AI assistant is not a generic chatbot. It is a **context-aware debugging assistant** designed specifically for coding errors.

Instead of allowing open-ended questions, the AI is triggered only when a user executes code and an error occurs.

When the code is run using the Judge API, if an error is returned, that error output is captured on the backend. I then send this error message along with the user's code as part of a structured prompt to the AI API.

The AI processes this context and returns a response explaining the error and possible fixes, which is then displayed to the user inside the room.

This makes the assistant more focused and relevant compared to a generic chatbot."

Q6: How does your code execution system work using Judge API? What happens from clicking "Run" to getting output?

"When a user clicks the Run button, the frontend sends the current code and selected programming language to the backend via a REST API.

The backend then formats this data and sends a request to the Judge API for execution.

Since code execution is asynchronous, the Judge API returns a submission ID initially. The backend then polls or waits for the execution result using that ID.

Once execution is complete, the Judge API returns the output, error, or status, which is then sent back to the frontend and displayed in the output panel."

- User clicks **Run**
- Frontend → sends code + language to backend
- Backend → calls Judge API
- Judge API → returns submission ID
- Backend → fetches result (polling)
- Backend → sends output/error to frontend

- UI updates

Q7: Why did you choose MongoDB for this project? Why not SQL?

“I chose MongoDB because my data structure is flexible and doesn’t require strict relational constraints.

For example, each room contains dynamic data like users and code, which can be easily stored as a document.

MongoDB’s document-based structure made it easier to store and update this data without designing complex schemas or joins.

Also, since this is a real-time application, I wanted faster development and easier scalability, which MongoDB supports well.

However, for applications with complex relationships or strict consistency requirements, SQL databases can be a better choice.”

One-line version (if short answer needed)

“I chose MongoDB for its flexible schema and ease of handling dynamic room-based data, while keeping in mind that SQL is better for structured relational systems.”

Q9: How did you manage state in your React frontend?

“In the frontend, I managed state using React’s useState and useEffect hooks.

I used useState to store dynamic data like the current code, connected users, selected language, and output.

useEffect was used to handle side effects like setting up and cleaning Socket.io event listeners.

For example, when a code update event is received from the server, I update the local state, which automatically re-renders the editor.

This allows the UI to stay synchronized in real time with minimal complexity.”

Q10: What challenges did you face while building this project and how did you solve them?

“One major challenge I faced was handling **concurrent edits** when multiple users type at the same time.

Initially, this caused conflicts where one user’s changes could overwrite another’s. I addressed this using a **last-write-wins strategy**, which simplified conflict handling for now.

However, I understand that this approach has limitations, and in more advanced systems, techniques like CRDT or Operational Transformation are used to handle this more effectively.

Another challenge I faced was implementing authentication with both traditional login and OAuth using Google OAuth.

The issue occurred when a user signed up using Google and later tried to log in using email/password, which caused failures since no password was stored.

I resolved this by distinguishing users based on their authentication provider and handling login flows accordingly, ensuring that OAuth users are authenticated only via Google.”

Q11: How did you ensure security in your application?

“I implemented authentication and session management to ensure only authorized users can access the platform.

For authentication, I used login-based access control where only authenticated users can create or join rooms.

I maintained sessions using tokens (like JWT), which are sent with each request and verified on the backend.

For authorization, I ensured that users can only access rooms they are part of by validating the roomId and user identity.

I also secured sensitive operations like code execution and API calls through the backend to prevent direct exposure of API keys.

Additionally, I used basic input validation to prevent invalid or malicious data from being processed.”

What is Session Management?

“Session management is the process of maintaining a user’s authenticated state across multiple requests after login.”

🎯 Q12: Disconnection Handling

“When a user disconnects, Socket.io automatically triggers a `disconnect` event.
On the server, I remove the user from the room and update the active users list.
Then I broadcast this update to the remaining users so the UI stays synchronized.”

🎯 Q13: Error Handling

“I implemented error handling at both frontend and backend levels.
On the backend, I used try-catch blocks and proper status codes.
On the frontend, I handled API failures and displayed user-friendly messages.
For Socket.io, I handled connection errors and reconnection events.”

🎯 Q14: Performance Optimization

“To improve performance, I minimized unnecessary updates by sending only relevant changes instead of full data.
I also avoided frequent re-renders in React and used efficient event handling for real-time updates.”

👉 Optional add:

“Debouncing can be used to further optimize frequent updates.”

AUTOSAVE-

“I implemented a debounced autosave mechanism for code persistence.
Instead of saving on every keystroke, the system waits until the user stops typing, and after a delay of 2 seconds, it triggers a save operation.

This reduces unnecessary database writes and improves performance while still ensuring that the latest code is persisted.”

🎯 Q15: Monaco Editor Integration

“I used Monaco Editor because it provides a professional coding experience with syntax highlighting and language support.
I integrated it as a React component and connected its change events with Socket.io to enable real-time syncing.”

🎯 Q16: Deployment

“I would deploy the frontend on platforms like Vercel or Netlify and the backend on a cloud server like AWS or Render.
For real-time support, I would ensure WebSocket compatibility and use environment variables to manage API keys securely.”

🎯 Q17: Rate Limiting / Abuse Prevention

“To prevent abuse, I would implement rate limiting on API endpoints and restrict excessive requests.
For code execution, I would limit execution time and resource usage to prevent misuse.”

🎯 Q18: Database Optimization

“To optimize database performance, I would use indexing on frequently queried fields like roomId.
I would also minimize unnecessary database writes and use caching if needed.”

🎯 Q19: Future Improvements

“I would improve the system by:

- implementing CRDT for better conflict resolution
- adding authentication improvements
- scaling using Redis for Socket.io
- enhancing the AI assistant for more use cases”